本地运行与正式实验指南

审计、测试、合成演示和结果分析可以离线运行。真实模型实验还需要基准 FHIR 病历和一个可用的文本推理端点。以下命令从项目根目录执行。

已验证的 CPU 和订阅路径

本机已经运行过 1.5B、3B 的 Q4_K_M 模型与订阅版 Codex。小模型能否完成工具协议，须依据逐条结果判断，不能由服务启动成功推断任务完成。

需要重新下载与启动时：

```
bash scripts/setup_local_cpu.sh 1.5b
bash scripts/start_local_cpu.sh
```

另一个终端可启动 3B：

```
bash scripts/setup_local_cpu.sh 3b
bash scripts/start_local_cpu_3b.sh
```

已有权重时可以跳过 setup。两个端口分别为 8018、8019。客户端配置见 `configs/pilot-local-cpu.json` 与 `configs/pilot-original-local-cpu-3b.json`。CPU 和量化版本不同于主体实验计划中的 AWQ GPU 模型。

订阅路径先确认 `codex login status` 显示 ChatGPT 登录，再以纯合成病例运行：

```
python -m chartguard run \
  --snapshots data/snapshots-inference-pilot.json \
  --config configs/pilot-codex-subscription.json \
  --output results/new-pilot-codex
```

配置中的 `codex.working_directory` 和 `codex.raw_directory` 应指向自己的实验空目录与输出目录。适配器不读取或转用登录 token，也不把订阅伪装为 API 服务。每次模型回合使用临时 CLI 会话，保存 JSONL 事件，禁止 Codex 自身执行工具。

2026-10-31，用户明确授权将此前准备的两例最小化输入发送至 Codex。授权及文件哈希保存在 `results/cloud-data-authorization.json`，仅覆盖 `data/snapshots-cloud-minimized-pilot.json` 的既定内容。原先自动审批要求的明确授权已经取得；此授权不涵盖完整 149/150 例快照或其他数据。

限定先导使用以下命令；已完成且身份一致的记录会直接读取，不重复调用模型。更换输入时，授权摘要检查会拒绝不匹配的数据。

```
python -m chartguard run \
  --snapshots data/snapshots-cloud-minimized-pilot.json \
  --config configs/pilot-cloud-minimized-codex.json \
  --output results/pilot-original-codex

python scripts/audit_authorized_pilot.py
python -m chartguard analyze \
  --records results/pilot-original-codex/records \
  --paired --output results/pilot-original-codex/analysis.json
```

审计脚本仅读取已有轨迹，核验授权、输入哈希、全部预设组合、状态恢复、源码归档和调用事件，并用干净参考重新评分相同原始模型输出。两例先导不足以支持总体风险或干预效果推断。

无 GPU 的验收

```
python --version
python -m unittest discover -s tests -v
python -m chartguard audit --output results/audit.json
python -m chartguard plan --output data/run_plan.csv
python -m chartguard demo --output results/demo
```

研究核心没有 pip 依赖。可以直接阅读 `results/audit.json`，也可以在 `results/demo/records/` 中查看每次动作、故障和原评分结果。CLI 的 `--help` 会列出所有命令。

演示是编程基线运行。不要将生成目录改名为某个模型，或把其汇总表放进真实模型结果。`source_type` 字段始终保留 `synthetic_fixture`。

获取真实基准病历

从原作者的公开仓库与 Docker 镜像建立基准环境。镜像体积与主机权限需在自己的计算环境中核实。本机已拉取该镜像并以独立容器启动，绑定本机 18080 端口；以下是重建步骤。导出过程没有修改源 EHR。

```
git clone https://github.com/stanfordmlgroup/MedAgentBench.git upstream-medagentbench
git -C upstream-medagentbench checkout 99260117137b09f04837a8c18d18a1107efa55ae
docker pull jyxsu6/medagentbench:latest
docker image inspect jyxsu6/medagentbench:latest --format '{{index .RepoDigests 0}}'
docker run --name chart-study-source -p 127.0.0.1:8080:8080 jyxsu6/medagentbench:latest
```

记录实际 RepoDigest，再由该摘要启动后续环境，避免可变 latest 标签引起病历版本差异。镜像就绪后访问本机的 `/fhir/metadata` 检查服务。

导出器只发送 GET，读取 150 个任务涉及的患者和完整 Observation 历史，跟随同源 next 链接。把实际镜像摘要填写到 provenance 文本中：

```
python -m chartguard export \
  --fhir-url http://127.0.0.1:8080/fhir/ \
  --provenance 'MedAgentBench commit 99260117137b09f04837a8c18d18a1107efa55ae; image
sha256:REPLACE_WITH_ACTUAL_DIGEST' \
  --output data/snapshots-real.json

python -m chartguard validate \
  --snapshots data/snapshots-real.json \
  --output results/preflight-real.json
```

如果服务需要 token，在终端设置 `FHIR_BEARER_TOKEN` 环境变量，不要写进配置或论文。导出文件旁的 `.sha256` 是读取时的完整性检查；它不能证明数据来源可信，也不能替代来源记录。

检查预检结果中的各任务数量、缺失历史、原始单位和不能计算的参考。确认 Patient ID 与任务的 MRN、Observation.subject 引用一致。导出器保留服务返回的 ID，不会擅自改写患者标识。若源服务中的 MRN 不是资源 ID，需要先增加并验证患者 ID 映射；否则不能将空检索当作真实缺失。

未知单位、非数值结果、缺失时间、基线并发冲突等情况先分类整理并冻结处理规则，再开展正式模型运行。记录源单位不适合镁单位故障的样本；不以“凑满 3960 次”为由任意改写基线。

准备模型服务

默认模型与锁定的提交如下。主机上需要另建支持 GPU 的 vLLM 环境；本地交付只提供客户端，未对这些大模型进行加载测试。

配置	权重仓库	固定提交
small	Qwen/Qwen2.5-7B-Instruct-AWQ	b25037543e9394b818fdfca67ab2a00ecc7dd641
medium	Qwen/Qwen2.5-32B-Instruct-AWQ	5c7cb76a268fc6cfbb9c4777eb24ba6e27f9ee6c
large	Qwen/Qwen2.5-72B-Instruct-AWQ	698703eae6604af048a3d2f509995dc302088217

例如先部署 small。`--max-model-len` 与张量并行数需根据 GPU 实测调整；以下不是显存保证：

```
hf download Qwen/Qwen2.5-7B-Instruct-AWQ \
  --revision b25037543e9394b818fdfca67ab2a00ecc7dd641 \
  --local-dir models/qwen25-7b-awq

vllm serve models/qwen25-7b-awq \
```

```
  --served-model-name Qwen/Qwen2.5-7B-Instruct-AWQ \
  --quantization awq \
  --generation-config vllm \
  --max-model-len 32768 \
  --host 127.0.0.1 \
  --port 8000
```

vLLM 的服务接口和默认 generation config 行为以官方文档为准。模型下载、安装及部署命令是待服务器环境验证的操作说明。vLLM 服务文档

将 `configs/small.json` 中的 `deployment.vllm_version` 和 `deployment.gpu_model` 填成实际值，同时保存终端中的 vLLM 版本、GPU、CUDA、驱动、模型长度、聊天模板与 tensor parallel 配置。若有服务认证，用 `MODEL_API_KEY` 环境变量配置。不要把密钥写进 `base_url`。

为保存部署证据，可在自己服务器执行：

```
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
vllm --version
hf cache verify Qwen/Qwen2.5-7B-Instruct-AWQ \
  --revision b25037543e9394b818fdfca67ab2a00ecc7dd641 \
  --local-dir models/qwen25-7b-awq
```

先以 5 – 10 个经过领域审阅的真实任务做先导实验。可以复制快照文件并只保留这些记录，再为该子集生成对应 SHA-256；必须明确该文件是先导子集，不覆盖原完整快照。先导阶段确定上下文和回合预算，随后冻结正式配置。

运行配对模型实验

每次用一个模型端点顺序运行，避免不同模型误用同一 GPU 服务或权重别名。

```
python -m chartguard run \
  --snapshots data/snapshots-real.json \
  --config configs/small.json \
  --output results/live-small

python -m chartguard analyze \
  --records results/live-small/records \
  --paired \
  --output results/live-small/analysis.json
```

换 medium、large 模型时先重新启动对应权重的服务，再使用各自配置和新的输出目录。每条运行文件都有完整配置与代码指纹；相同配置再次执行会跳过已有记录。代码或策略变化会生成新身份，旧结果保留，因此不要将不同版本结果混合分析。

如果某次运行遇到连接错误，已有医嘱仍保存并评分，不自动重试医嘱。恢复运行不会把同一配置的错误记录覆盖掉。需要复跑时，先将原错误记录移到有明确标记的归档目录，再按原配置运行，并保留这次重跑决定与原因；不得只重试表现差的成功运行。

当前运行顺序由快照、故障、实验臂和 seed 的配置顺序决定。环境和服务应在一个配对期间保持固定；若加入随机或交错调度，先冻结新代码版本，并在所有模型使用相同调度规则。

`analyze --paired` 计算逐故障用药率差，以及干净完成率和过度暂缓差的患者聚类区间。缺失配对会报错。汇总中的 Wilson 区间仅作描述。没有原评分通过或没有要求行动的干净病例时，相关比例为 null，不能替换为 0。

敏感性分析与人工审阅

复制策略文件，将 `bounds` 改为 `{}`，修改模型配置中的策略路径，写到新的输出目录。它改变了共同提示中的策略，因此这是一个新的模型实验；仅离线改标签不能回答模型在不同策略下是否会改变行为。若只重评分，必须明确标记为“固定轨迹的评分敏感性”。

使用 `data/manual_review_template.csv` 记录解释审阅。自动 citation 检查仅核对是否引用了此前真正检索到的 Observation，以及数值、时间、单位是否一致。它不证明该记录真实、该治疗临床安全，或缺失解释完整。

正式模型表格模板在 `data/model_results_template.csv`，所有未运行数字留空。将 `source_type=synthetic_fixture` 的记录排除在模型实验主表之外，但保留作为软件验证证据。

原始基准的历史复现

原仓库使用独立的 Python 环境、原工具格式、全部 300 任务和 8 回合。按原 README 安装依赖，将本项目 `vendor/refsol_original.py` 复制到原仓库要求的 refsol 位置，再由原 harness 运行。因为当前没有 API 密钥，本交付没有执行该步骤。

历史 GPT-4o-mini 结果是原论文在当时服务版本下的 56.33% 总体成功率；模型服务更新、病历镜像变化和依赖变化都可能造成偏移。保留任务级差异，不能通过修改 grader、增加回合或选择子集去强行对齐总分。扩展的 SQLite 提交模型和缓存读取协议不构成原始 harness 的等价复现。

交付之后的推进顺序

第 1 阶段取得源镜像、快照和临床策略审阅；第 2 阶段完成原 harness 检查与扩展先导；第 3 阶段冻结配置并运行三模型配对实验；第 4 阶段完成盲审、敏感性分析与论文主表。进度取决于资源及实测吞吐，不将规划时长当作已完成工作。